

Node.js Interview Preparation Guide

1. Node.js Fundamentals

- **What is Node.js?** A JavaScript runtime built on Chrome's V8 engine that executes JS outside the browser, using an event-driven, non-blocking I/O model.
- **Single-threaded but non-blocking:** Node runs your JS on a single main thread, but I/O operations (file, network, DB) are offloaded to the OS/libuv thread pool, so the main thread isn't blocked waiting.
- **libuv:** The C library providing the event loop, thread pool, and async I/O to Node — this is what makes non-blocking I/O possible across platforms.
- **When NOT to use Node:** CPU-intensive tasks (heavy computation, image/video processing) block the single thread — better handled via worker threads, clustering, or offloading to another service.

Common questions: Is Node.js single-threaded? (The event loop is single-threaded; libuv's thread pool and some OS async operations are not.) Why is Node.js good for I/O-heavy apps but not CPU-heavy ones?

2. Event Loop (very important, highly asked)

- The mechanism that lets Node perform non-blocking I/O by offloading operations and processing callbacks when ready.
- **Phases (in order):**
 - Timers** — `setTimeout`, `setInterval` callbacks.
 - Pending callbacks** — I/O callbacks deferred from previous cycle.
 - Idle, prepare** — internal use.
 - Poll** — retrieve new I/O events, execute I/O-related callbacks.
 - Check** — `setImmediate()` callbacks.
 - Close callbacks** — e.g., `socket.on('close', ...)`.
- **Microtasks** (`process.nextTick`, resolved Promises) run **between every phase**, and `process.nextTick` has higher priority than Promise microtasks.

```
1 console.log('start');
2 setTimeout(() => console.log('timeout'), 0);
3 setImmediate(() => console.log('immediate'));
4 process.nextTick(() => console.log('nextTick'));
5 Promise.resolve().then(() => console.log('promise'));
6 console.log('end');
7 // Output: start, end, nextTick, promise, timeout/immediate (order of last two can vary)
```

Common questions: Difference between `setImmediate` and `setTimeout(fn, 0)`? Why does

`process.nextTick` run before Promises? What happens if you block the event loop with a synchronous heavy loop?

3. Modules: CommonJS vs ES Modules

	CommonJS	ES Modules
Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Loading	Synchronous	Asynchronous (statically analyzed)
File extension	<code>.js</code> (default)	<code>.mjs</code> or <code>"type": "module"</code> in <code>package.json</code>
Top-level await	Not supported	Supported
Tree-shaking	Not possible (dynamic)	Possible (static structure)

```
1 // CommonJS
2 const fs = require('fs');
3 module.exports = { readData };
4
5 // ESM
6 import fs from 'fs';
7 export function readData() {}
```

Common questions: Can you mix CJS and ESM? How does `require` caching work (modules cached after first load)?

4. Asynchronous Programming

Callbacks

- Classic Node pattern: `fn(err, data) => {}` (error-first callback convention).
- Problem: **callback hell** — deeply nested callbacks reducing readability.

Promises

- Represent a value that will resolve/reject in the future; chainable via `.then/.catch/.finally`.

`async/await`

- Syntactic sugar over Promises, making async code read like sync code.

```

1  async function getUser(id) {
2    try {
3      const res = await db.findUser(id);
4      return res;
5    } catch (err) {
6      logger.error(err);
7      throw err;
8    }
9  }

```

Common questions: How do you run promises in parallel? (`Promise.all` , `Promise.allSettled`) Difference between `Promise.all` and `Promise.race` ? How do you handle unhandled promise rejections globally (`process.on('unhandledRejection')`)?

5. Streams & Buffers

Streams

- Handle reading/writing data in **chunks** instead of loading everything into memory — critical for large files, video, network I/O.
- Types: **Readable**, **Writable**, **Duplex** (both), **Transform** (modifies data as it passes through, e.g., gzip).

```

1  const fs = require('fs');
2  const readStream = fs.createReadStream('large.txt');
3  const writeStream = fs.createWriteStream('copy.txt');
4  readStream.pipe(writeStream); // handles backpressure automatically

```

- **Backpressure:** when a writable stream can't keep up with a readable stream — `.pipe()` handles this automatically by pausing/resuming.

Buffer

- Raw binary data storage (fixed-size, outside V8's heap) — used when working with streams, TCP, file I/O, or binary protocols.

```

1  const buf = Buffer.from('hello', 'utf-8');

```

Common questions: Why use streams instead of reading a whole file into memory? What is backpressure and how is it handled?

6. Express.js & Middleware

- **Express** is the most common minimal web framework for Node, providing routing, middleware pipeline, and request/response helpers.
- **Middleware:** functions with signature `(req, res, next)` that run in sequence for each request — used for logging, auth, parsing body, error handling.

```

1  const express = require('express');
2  const app = express();
3
4  app.use(express.json());           // body parser middleware
5  app.use((req, res, next) => {      // logging middleware
6    console.log(`${req.method} ${req.url}`);
7    next();
8  });
9
10 app.get('/users/:id', (req, res) => {
11   res.json({ id: req.params.id });
12 });
13
14 // Error-handling middleware (4 args – must come last)
15 app.use((err, req, res, next) => {
16   res.status(500).json({ error: err.message });
17 });

```

Common questions: Order of middleware execution matters — why? Difference between `app.use` and `app.get` ? How do you write reusable middleware (e.g., auth check, validation)?

7. REST API Design & Error Handling

- **REST principles:** resource-based URLs (`/users/:id`), correct HTTP verbs (GET/POST/PUT/PATCH/DELETE), stateless requests, proper status codes (200, 201, 400, 401, 403, 404, 500).
- **Error handling patterns:**
 - Centralized error-handling middleware (Express) that maps custom error classes to status codes.
 - Custom error classes: `class NotFoundError extends Error {}` for consistent, typed errors.
 - Always validate input (e.g., with `Joi` / `zod`) before hitting business logic.
 - Wrap async route handlers to avoid unhandled promise rejections (a small `asyncHandler` wrapper or `express-async-errors`).

Common questions: How do you design pagination/filtering/sorting for a list endpoint? How do you version an API (`/v1/` , header-based)? How would you structure validation, controller, service, and repository layers?

8. Scaling Node: Clustering, Child Processes, Worker Threads

Cluster module

- Node runs on a single core by default; `cluster` forks multiple worker processes (one per CPU core) sharing the same server port to utilize multi-core machines.

```
1  const cluster = require('cluster');
2  const os = require('os');
3
4  if (cluster.isPrimary) {
5    os.cpus().forEach(() => cluster.fork());
6  } else {
7    require('./server'); // each worker runs the app
8  }
```

Child Processes

- `child_process.spawn/exec/fork` — run separate OS processes (e.g., run a shell command, another Node script) with their own memory space; communicate via IPC.

Worker Threads

- `worker_threads` module — true multi-threading within a single Node process, sharing memory via `SharedArrayBuffer` — ideal for CPU-bound tasks (image processing, heavy calculations) without blocking the event loop.

Common questions: Cluster vs Worker Threads vs Child Process — when to use which? How does load balancing work across cluster workers (round-robin by default on Linux)?

9. Databases & ORMs

- **SQL (PostgreSQL/MySQL)** via `pg`, `mysql2`, or ORMs like **Sequelize/Prisma** — schema-enforced, relational, ACID transactions.
- **NoSQL (MongoDB)** via **Mongoose** — flexible schema, document-based, good for rapidly evolving data models.

```
1  // Mongoose example
2  const userSchema = new mongoose.Schema({ name: String, email: { type: String, unique: true } });
3  const User = mongoose.model('User', userSchema);
4  await User.create({ name: 'Alex', email: 'a@x.com' });
```

- **Connection pooling:** reuse DB connections instead of opening a new one per request (critical for performance under load).
- **Transactions:** ensure multi-step operations are atomic (`session.startTransaction()` in Mongoose, or

SQL BEGIN/COMMIT).

Common questions: SQL vs NoSQL — when would you pick one over the other? How do you prevent N+1 query problems? How do you handle DB migrations?

10. Authentication & Authorization

- **JWT (JSON Web Tokens):** stateless auth — server signs a token containing claims; client sends it in `Authorization: Bearer <token>` header; server verifies signature, no session store needed.

```
1  const token = jwt.sign({ userId: user.id, role: user.role }, SECRET, { expiresIn: '1h' });
2  // Middleware
3  function auth(req, res, next) {
4    const token = req.headers.authorization?.split(' ')[1];
5    try {
6      req.user = jwt.verify(token, SECRET);
7      next();
8    } catch {
9      res.status(401).json({ error: 'Invalid token' });
10   }
11 }
```

- **Sessions + cookies:** stateful auth — server stores session data (in-memory/Redis), client holds a session ID cookie.
- **RBAC (Role-Based Access Control):** middleware checks `req.user.role` against allowed roles for a route.
- **Password hashing:** always hash with `bcrypt` (salted, slow by design) — never store plaintext passwords.

Common questions: JWT vs sessions — trade-offs (statelessness, revocation difficulty with JWT)? How do you handle token refresh (refresh tokens, short-lived access tokens)? How do you securely store JWTs on the client (httpOnly cookies vs localStorage)?

11. Security

- **Helmet:** sets secure HTTP headers (CSP, X-Frame-Options, etc.) automatically.
- **CORS:** configure allowed origins explicitly rather than `*` in production.
- **Input validation/sanitization:** prevent NoSQL/SQL injection (`express-validator` , parameterized queries, avoid string-concatenated queries).
- **Rate limiting:** `express-rate-limit` to prevent brute-force/DDoS on sensitive endpoints (login, OTP).
- **Environment variables:** secrets in `.env` (never committed), loaded via `dotenv` , injected via deployment platform secrets in production.

- **HTTPS/TLS:** always enforce in production; redirect HTTP → HTTPS.
- **Dependency audits:** `npm audit`, keep packages patched.

Common questions: How do you prevent SQL/NoSQL injection? How do you secure an API against brute-force login attempts? What does Helmet actually do under the hood?

12. Caching

- **Redis** — in-memory key-value store used for: session storage, caching expensive DB query results, rate-limiting counters, pub/sub messaging, distributed locks.

```
1  const cached = await redisClient.get(`user:${id}`);
2  if (cached) return JSON.parse(cached);
3  const user = await db.findUser(id);
4  await redisClient.set(`user:${id}`, JSON.stringify(user), 'EX', 3600);
```

- **Cache invalidation strategies:** TTL expiry, write-through, cache-aside (lazy loading, shown above).

Common questions: Cache-aside vs write-through vs write-behind? How do you avoid stale cache/cache stampede issues?

13. Testing

- **Unit testing:** Jest/Mocha + Chai — test pure functions, services, controllers in isolation with mocked dependencies (DB, external APIs via `jest.mock` or `sinon`).
- **Integration testing:** `supertest` to hit actual Express routes and assert responses, often against a test DB (or in-memory Mongo via `mongodb-memory-server`).

```
1  const request = require('supertest');
2  test('GET /users/:id returns user', async () => {
3    const res = await request(app).get('/users/1');
4    expect(res.status).toBe(200);
5    expect(res.body).toHaveProperty('id', '1');
6  });
```

- **Mocking:** mock external services (payment gateways, third-party APIs) so tests are fast and deterministic.

Interview hack: Mention structuring code in layers (controller → service → repository) specifically so each layer is unit-testable independently — signals mature backend design thinking.

14. Performance & Optimization

- **Avoid blocking the event loop:** move heavy sync computation to worker threads or a separate service.
 - **Connection pooling** for DB clients.
 - **Compression:** `compression` middleware (gzip/Brotli) for responses.
 - **Pagination** for large list endpoints instead of returning everything.
 - **Caching** hot paths (Redis, HTTP caching headers/ETags).
 - **Clustering / horizontal scaling** behind a load balancer.
 - **Profiling tools:** `node --prof`, Clinic.js, Chrome DevTools (`node --inspect`) to find bottlenecks.
 - **Logging:** structured logging (`winston` / `pino`) with log levels, avoid `console.log` in production at scale.
-

15. Microservices & Architecture

- Splitting a monolith into independently deployable services communicating via REST/gRPC/message queues (RabbitMQ, Kafka).
- **API Gateway** pattern — single entry point routing to services, handling auth/rate-limiting centrally.
- **Message queues:** decouple services, handle async workflows (e.g., order placed → queue → email service consumes it).
- **Service discovery, circuit breakers** (e.g., `opossum`) for resilience when a dependent service is down.

Common questions: Monolith vs microservices — trade-offs? How do services communicate reliably (sync vs async, retries, idempotency)? How do you handle distributed transactions (saga pattern)?

16. npm, package.json & Environment

- **package.json** : dependencies vs devDependencies, `scripts` , semantic versioning (`^` , `~`).
 - **package-lock.json** : locks exact dependency tree for reproducible installs.
 - **Environment configs:** `.env` files per environment (dev/staging/prod), never hardcode secrets.
 - **Process managers:** PM2 for keeping a Node process alive, zero-downtime reloads, clustering built-in.
 - **Docker:** containerize the app for consistent deployment across environments; multi-stage builds to keep images small.
-

17. Debugging & Monitoring

- `node --inspect` + Chrome DevTools for breakpoints/profiling.
- `console.trace` , structured logging with correlation/request IDs for tracing requests across services.
- APM tools (New Relic, Datadog) for latency/error monitoring in production.
- Health-check endpoints (`/health`) for load balancers/orchestrators (Kubernetes liveness/readiness probes).

Quick Final Tips

- Be ready to **write live code** for: an Express route with middleware + error handling, a JWT auth middleware, a rate limiter, a stream-based file processor, or a small `cluster` example.
- Always explain **why**, not just what: e.g., "I used a cache-aside pattern with Redis to cut DB load on a hot read path."
- Have one real project story ready: scaling an API (clustering/caching), fixing an event-loop-blocking bug, or securing an auth flow — concrete metrics (latency drop, requests/sec) land well.